

Feature: Who does what

From build-plan: feature 5b

Branch: feature/05b-who-does-what

Status: verified. Whole feature seen and agreed by Frank 2026-10-01; steps 5b.1 to 5b.3 built, tested and reviewed step by step; every review finding fixed, F-57 last on 2026-10-01. The checkpoint for the final review.

Branch named in the workspace's `feature/NN-name` form, so one number finds the branch, the archive (`05b-who-does-what.md`) and the tag (`item-05b-done`). The second of feature 5's four parts (split 2026-10-01, Frank).

Goal

The data the free-time check (5c) reads to know who may be offered for a service: which people can do it (skills), which places it can be done in (rooms), and which people are on standby on a given date (at work, hidden from customers, still placeable by the owner). Project plan decision 26: who does what is skills and rooms, checked at booking and ticked in settings; nobody ticked means anyone can do it, no room ticked means no room check.

Nothing is visible yet. The owner ticks skills and rooms and sets standby on screen in feature 12; until then the dev seed and the tests write the rows.

In scope

- `booking_link_resource` : one row per tick, a person who can do a service or a place it can be done in. Migration 0010, which also gives `booking_link` a unique `(organizationId, id)` so the tick can only name a service of its own business.
- `standby_date` : one row per person per date they are on standby. Migration 0011.
- `findServiceResources` and `findStandbyDates` in `backend/lib/scheduling/` , the two reads 5c calls.
- The dev seed: Riverbend Clinic (dev) gets skills, rooms and one standby date, shaped like Face and Body, so 5c has real cases to work on.
- F-53's one-line fix (a finding number in a comment of `backend/lib/scheduling/hold-time.test.ts`) rides on step 5b.1's commit.

Out of scope

- Free times, "any available", buffers (5c). Booking (5d).

- Writing ticks or standby from a screen or a route (feature 12, 12b): these are tables and backend reads, as in 5a.
- A service that needs two people or two places at once (named, not planned).
- Part-day standby: the plan says standby is dated (see Decisions).
- Applying the migrations to Railway (at deploy, Frank's line).

Build loop

`workflow.stepReview` is `every` and `workflow.checkpointCommits` is `enabled`.

Before the first step Frank sees the whole feature once. Each step's plan (Part 1, what it builds; Part 2, Done when) is gone through with him and gets its own yes just before it is built. After that yes the step runs straight through without asking: build, tests and checks, tick the box here, write the build log entry and push buildlogs, commit (`feat: 5b.N <what>`) and push to the feature branch, `/audit` scoped to the step, then the independent review. The one planned stop is after the review, where its findings are talked through. P0/P1 are fixed before the next step, or Frank accepts them with a reason; P2/P3 are recorded and carried. A finding that only makes the agreed plan hold is fixed without asking.

Only three things stop a step earlier: the agreed plan turns out wrong while building, a line only Frank crosses (a package, Railway or real data, `main` , a merge, a force push, deleting anything), or blocking review findings.

`/complete` runs its own final review over the already-reviewed steps, then merges with a merge commit on Frank's yes.

Build steps

- ☒ **5b.1 Skills and rooms.**
 - `packages/shared/db/scheduling-tables/booking-link-resource-table.ts` ; `booking_link` gains `unique("booking_link_organization_id_unique")` on `(organizationId, id)` ; migration `0010_booking_link_resource` , generated.
 - The dev seed ticks Riverbend Clinic's services (below), idempotent like the rest of the seed.
 - F-53: the finding number leaves the comment in `hold-time.test.ts` .
 - **Done when** backend tests prove, by inserting rows directly: a person and a place can be ticked for a service; the same tick twice is refused by the primary key; a tick cannot name another business's service or another business's person or place (both composite foreign keys, by name); deleting a service takes its ticks; a ticked person or place cannot be deleted until unticked (F-54); deleting the business takes everything; `db:migrate` applies cleanly and `db:seed` runs twice without changing anything the second time. The backend builds.
- ☒ **5b.2 Standby dates.**

- `packages/shared/db/scheduling-tables/standby-date-table.ts` ; migration `0011_standby_date` , generated.
- The dev seed puts one clinic practitioner on standby on one date.
- **Done when** backend tests prove: a person can be on standby on a date; the same person twice on one date is refused by name; two people on one date are fine; another business's person is refused by the composite foreign key; deleting the person or the business takes the rows; `db:migrate` and `db:seed` as in 5b.1. The backend builds.
- ☒ **5b.3 The two reads 5c calls.**
 - `backend/lib/scheduling/find-service-resources.ts` and `backend/lib/scheduling/find-standby-dates.ts` , tests beside each.
 - **Done when** backend tests prove `findServiceResources` : with nobody ticked, every active person of the business, and no room check; with people ticked, only those, inactive ones left out, so all ticked people inactive gives nobody (never "anyone"); with places ticked, only the active ones, and all inactive gives none (the service cannot be placed); a person ticked never shows as a place or the reverse; an inactive, missing or other business's service answers `null` . And `findStandbyDates` : exactly the asked people's dates inside the range, both ends included, never another business's. No error carries more than a safe reason. The backend builds.

Files / areas

- `packages/shared/db/scheduling-tables/` (two new tables), `packages/shared/db/booking-tables/booking-link-table.ts` (the unique), `packages/shared/db/index.ts`
- `packages/shared/migrations/0010_booking_link_resource.sql` , `0011_standby_date.sql` , generated from `packages/shared`
- `packages/shared/scripts/seed-dev.ts`
- `backend/lib/scheduling/` : two reads, tests beside them, and a test file for each table's rules, as `commitment-rules.test.ts` does

Data / contracts

booking_link_resource (who does what)

- `organizationId` (text, FK organization, cascade), `bookingLinkId` (text), `resourceId` (text), `createdAt` .
- Primary key (`bookingLinkId`, `resourceId`) : a tick exists or not.
- FK (`organizationId`, `bookingLinkId`) to `booking_link(organizationId, id)` , cascade, named `booking_link_resource_booking_link_fk` ; FK (`organizationId`, `resourceId`) to `resource(organizationId, id)` , **no action**, named `booking_link_resource_resource_fk` . A deleted service takes its ticks; a ticked person or place cannot be deleted until unticked (F-54): losing

the only tick would turn "only them" into "anyone", the very answer an inactive tick is refused (5b.3). Deleting the business still clears everything.

- The resource's own `kind` says what the tick means: a person can do the service, a place is a room it can be done in. Any one ticked person and any one ticked place is enough for one booking.

standby_date

- `organizationId` (text, FK organization, cascade), `resourceId` (text), `date` (Postgres `date`, read as `YYYY-MM-DD` text, the business's own calendar date), `createdAt`.
- Primary key (`resourceId`, `date`), named `standby_date_pkey`.
- FK (`organizationId`, `resourceId`) to `resource(organizationId, id)`, cascade, named `standby_date_resource_fk`.
- Meaning, read by 5c: on that date the person is hidden from customers. The owner can still place them; off is time off (a `commitment`, 5a). The plan says per person; the table does not refuse a place, which 5c reads the same way (a room hidden from customers that day).

findServiceResources(organizationId, bookingLinkId) answers `null` when the service is missing, inactive or another business's; otherwise `{ peopleIds: string[], placeIds: string[] | null }`:

- `peopleIds`: the ticked people who are active; with no person ticked, every active person of the business. Ordered by name, then id.
- `placeIds`: the ticked places that are active, ordered the same way; `null` when no place is ticked (no room check). An empty array means a room is needed and none can be used.

findStandbyDates(organizationId, resourceIds, fromDate, toDate) answers `{ resourceId, date }[]` for those resources with `fromDate <= date <= toDate` (`YYYY-MM-DD` strings), ordered by date then resource. An empty list of resources answers `[]` without a query.

Failures in both throw `Reading who does what failed: <safe reason>` and `Reading standby failed: <safe reason>`, through `safeErrorReason`.

Dev seed, Riverbend Clinic (dev)

- Facials (Deep Cleansing, Dermaplaning, Hydra Spa): Sofia, Ana, Mei; Room 3.
- Chemical Peel: nobody ticked (anyone); Room 3.
- Massages (the four): Luis, Ana, Priya, Daniel; Rooms 1 to 4.
- Laser Hair Removal: Mei; Room 5.
- Body Wrap: Priya; Room 4.
- Every clinic service needs a room, so the "no room check" case in dev is Summit Painting (dev), whose service ticks nothing.
- Standby: Sofia on the first Monday at least seven days ahead of the seed run, a date she would otherwise be bookable.

Testing

Backend tests against the local `scheduleads_dev` with the existing guard, each test making its own throwaway business, people, places and services so it passes on its own. Database rules are proved by expecting Postgres to refuse the bad row with the named constraint. Each new test shown able to fail once, by breaking the rule or planting a fault. The seed's second run is checked by counting rows before and after. No frontend change.

Notes for the AI

- `booking_link` has no unique `(organizationId, id)` yet; the composite FK needs it, as `resource_organization_id_unique` does for `resource`. Check `db:generate` writes it before the new table's FK in 0010.
- Drizzle's `date` column with `mode: "string"` keeps `YYYY-MM-DD` and never shifts a date through a time zone.
- A later `db:generate` must find no changes after 0011.
- Seed: reuse its find-or-make pattern; insert ticks and standby with `onConflictDoNothing` so a second run changes nothing. The standby date is computed from the run date, like `sundayAfterDays`.
- Function names: `findServiceResources` reads the words of the table (`booking_link_resource`); "who does what" is the feature's name, not a code word.

Implementation walkthrough

What was actually built, by area. The steps above are the plan; the build log records how each step went against it. This is the shape of the code the feature left behind, and the decisions in it that are not visible from the code alone.

packages/shared

- `db/scheduling-tables/booking-link-resource-table.ts` : who does what, one row per tick, a service and a person or place; the resource's own `kind` says whether the tick means "can do it" or "is done here". The primary key `(bookingLinkId, resourceId)` makes a tick exist or not. Two composite foreign keys keep a tick inside its business. Deleting a service takes its ticks (cascade); a ticked person or place cannot be deleted until unticked (no action, F-54), because losing the only tick would turn "only them" into "anyone". Deleting the whole business still clears everything, since `no action` is checked at the end of that one statement.
- `db/booking-tables/booking-link-table.ts` gained `booking_link_organization_id_unique` on `(organizationId, id)`, only so a tick can point at a service of its own business.
- Migration `0010_booking_link_resource.sql` : generated, then one statement moved by hand to the top. `drizzle-kit` writes the new unique on `booking_link` after the foreign key that needs it, and Postgres refuses that order (`42830`). A later `db:generate` finds no change. 0010 was rebuilt in place

once, for F-54, while it had only been applied on the laptop (F-56, accepted: dev databases are rebuilt freely).

- `db/scheduling-tables/standby-date-table.ts` : one row per person per date they are on standby (at work, hidden from customers, still placeable by the owner). `date` is a Postgres `date` read in Drizzle's string mode, so it is the business's calendar date as `YYYY-MM-DD` and no time zone can shift it. Primary key `standby_date_pkey (resourceId, date)` ; it goes with the person (cascade), which cannot widen anything. Migration `0011_standby_date.sql` , generated as it was.
- `scripts/seed-dev.ts` : Riverbend Clinic's services ticked like Face and Body's (49 ticks; the peel ticks only its room, so anyone can do it) and Sofia on standby on the first Monday at least a week after the seed runs. Ticks and dates are inserted with `onConflictDoNothing` , so a second run changes nothing; a ticked name the business does not have throws and the whole seed rolls back.

backend/lib/scheduling

- `find-service-resources.ts` : who may be offered for a service. `null` for a missing, inactive or other business's service. The ticks are read with their resource's `kind` and `active` (the join compares the business too), then split: the active ticked people, or every active person of the business when no person is ticked; the active ticked places, or `null` (no room check) when no place is ticked. Ticked but all inactive gives an empty list, never "anyone". Both lists ordered by name, then id. Two queries, three when nobody is ticked.
- `find-standby-dates.ts` : the standby dates of some people over an inclusive range of `YYYY-MM-DD` dates, ordered by date then person, inside the business. No people asked answers `[]` without a query.
- Tests: a rule test per table proving each constraint by name, and one test file beside each read. Every test was shown able to fail: rules broken on the local database for the tables, fourteen planted faults for the reads. F-57 added a standby date just before the tested range, because a planted fault that made the lower bound stricter had not proved the bound exists.

Alongside the feature

- Every built feature now has a Play it explorable and a Knowledge drawer in the build log (features 1, 2, 3, 3b, 4, 5a and 5b), on the shared theme. 5b's, "Who gets offered", models exactly the rule `findServiceResources` implements.

What later features inherit

- 5c starts its free times from `findServiceResources` (people, and rooms or no room check) and hides people on `findStandbyDates` , then subtracts `findCommitments` (5a) and Google busy times.
- 5d checks the same answer again before booking.
- Feature 12's settings screen ticks people and rooms and sets standby; it must untick a person before deleting them (F-54), and decides how.

Findings

Resolved during this feature and archived with it. IDs carry the feature number; the bare IDs inside each entry are the ones used while the work was live. F-32, F-47 and F-52 stay in the live ledger; F-56 was accepted by Frank.

5b/F-53 [P3] closed - The deadlock test's comment carries a finding number, which the standards keep out of code

File: backend/lib/scheduling/hold-time.test.ts:111 **Found:** 2026-10-01 by /audit independent (scope: current, c01dd9c..da22890; lens: quality) **Why it matters:** The comment above the stress test reads "can deadlock inside Postgres (F-51)". `coding-standards.md` (Comments, the balance) says no history in code comments, naming "finding numbers" explicitly: that lives in the build log. It is the only finding number in backend or shared code apart from the older migration 0000, and test files are the pattern later steps copy (4/F-53 and 4/F-54 were exactly that). Once `/complete` archives this ledger, a bare `F-51` in code points at nothing. **Suggested fix:** Drop "(F-51)" from the comment; the sentence already says why the test exists. **Resolution:** Fixed 2026-10-01 in step 5b.1: the comment above the deadlock test no longer names a finding. Closed 2026-10-01 by the step 5b.1 review (059cf0f..27d3d52): line 111 now reads "can deadlock inside Postgres; one is killed", no finding number is left, and the one-line change introduced nothing new in the file.

5b/F-54 [P2] closed - Deleting the last ticked person or room quietly opens a service to anyone, or drops its room check

File: packages/shared/db/scheduling-tables/booking-link-resource-table.ts:30 **Found:** 2026-10-01 by /audit (scope: step 5b.1, 059cf0f..27d3d52; lens: quality) **Why it matters:** Both foreign keys cascade, as the spec says ("ticks are settings, not history"), and the rules test proves deleting a person takes their ticks. But the rule 5b.3 reads is "nobody ticked means anyone, no place ticked means no room check". So when Laser Hair Removal ticks only Mei and Room 5, deleting Mei (a person with no bookings yet, which `commitment` does not block) leaves no person ticked and any practitioner is offered; deleting Room 5 leaves no place ticked and laser is booked with no room at all. 5b.3 deliberately says all ticked people inactive gives nobody, "never anyone"; deleting reaches exactly the "anyone" answer it rules out for deactivating. Nothing deletes a person or a service outside tests today, so it becomes reachable with feature 12's settings screen. **Suggested fix:** Decide before 5b.3 builds on it, with Frank. Smallest options: make `booking_link_resource_resource_fk` `no action`, so a ticked person or room cannot be deleted and is deactivated instead, as `commitment` already forces (0010 is applied nowhere real, so it can be regenerated); or keep cascade and write into feature 12 that removing someone refuses, or warns, when it would leave a service with no person or no room ticked. **Resolution:** Fixed 2026-10-01 in 5b.1's review-fix commit, without asking, since it only makes the agreed rule hold (ticked but gone means nobody, never anyone): `booking_link_resource_resource_fk` is now `no action`, edited into 0010 (local only) and rebuilt. A ticked person or place cannot be deleted until unticked; a deleted service still

takes its ticks; deleting the business still clears everything. The test "a ticked person or room cannot be deleted until unticked" failed against the cascade and passes now. Spec corrected. Feature 12 decides how Settings removes a ticked person (untick first, or deactivate). Closed 2026-10-01 by the step 5b.2 review (27d3d52..0c90b64): the table, 0010 and the 0010 snapshot all say `no action`, and the local database holds it (`confdeltype a`). A rolled-back probe confirmed a ticked person is refused by `booking_link_resource_resource_fk` (their standby rows stay), and deleting the business in one statement still clears resource, ticks and standby rows. All migrations 0000 to 0011 apply cleanly to a fresh schema (rolled back). `db:generate` reports no changes. Nothing new introduced. (This resolution and F-55's had been written into F-32's Resolution by mistake; moved here, F-32 restored.)

5b/F-55 [P3] closed - The new table's header comment names step 5b.3, which the standards keep out of code

File: packages/shared/db/scheduling-tables/booking-link-resource-table.ts:3 **Found:** 2026-10-01 by /audit (scope: step 5b.1, 059cf0f..27d3d52; lens: quality) **Why it matters:** The header ends "(decided where it is read, 5b.3)". `coding-standards.md` (Comments, the balance) keeps step numbers out of code comments, the same rule F-53 was fixed for in this very commit. Once the feature is archived, "5b.3" points at nothing; the function name would not. **Suggested fix:** Say "(decided where it is read, findServiceResources)", or drop the parenthesis. **Resolution:** Fixed 2026-10-01: the header names `findServiceResources` instead of a step. The same commit also clears the older "step 3.3" comment the reviewer saw in `backend/lib/calendar/save-calendar-connection.ts`. Closed 2026-10-01 by the step 5b.2 review (27d3d52..0c90b64): line 3 now ends "(findServiceResources reads it)", and `save-calendar-connection.ts:38` names `getBusyTimes`, which does set `lastCheckedAt` (`get-busy-times.ts:93`). No step or finding number is left in any comment the range adds, and nothing new was introduced.

5b/F-56 [P3] accepted - A database that applied the first 0010 fails on db:migrate after the regenerated one

File: packages/shared/migrations/meta/_journal.json:78 **Found:** 2026-10-01 by /audit (scope: step 5b.2, 27d3d52..0c90b64; lens: quality) **Why it matters:** 0010 was regenerated in place, so its journal `when` moved from 1790903639726 to 1790904391831. The migrator runs every migration whose `when` is later than the newest `created_at` in the ledger (drizzle-orm 0.45.2 `pg-core/dialect.js:62`). A database that applied 27d3d52's 0010 (pushed to the feature branch) therefore re-runs 0010 on the next `db:migrate`, fails at the unique that already exists, and never reaches 0011. This laptop's `scheduleads_dev` was repaired by hand (its ledger holds the new 0010 and 0011 hashes), and Railway never had 0010. Not observed elsewhere; it bites only if another machine migrated between the two commits. **Suggested fix:** Nothing in code. On any other machine that migrated in that window, rebuild the dev database (or drop `booking_link_resource`, `booking_link_organization_id_unique` and the 0010 ledger row), then `db:migrate` and `db:seed`. **Resolution:** Accepted by Frank, 2026-10-01: dev databases are disposable for him; he will rebuild the dev database (migrate and seed from scratch) when

he next works on another machine, in about a month, so nothing will apply the old 0010. Railway never had it.

5b/F-57 [P2] closed - No standby test puts a date before the range, so a dropped lower bound passes every test

File: backend/lib/scheduling/find-standby-dates.test.ts:55 **Found:** 2026-10-01 by /audit (scope: step 5b.3, 3db43e2..a96ee29; lens: tests) **Why it matters:** The Done when asks the tests to prove "exactly the asked people's dates inside the range, both ends included". The fixture's earliest date is 2026-10-12 (line 39) and the range test starts on 2026-10-12 (line 55); the other tests ask for 2026-10-01 to 2026-10-31. No row ever sits before `fromDate`, so deleting `gte(standbyDate.date, fromDate)` at find-standby-dates.ts:29 leaves all four tests green. The upper end is proven (2026-10-26 is left out of a range ending 2026-10-19), the lower end only for inclusion. 5c reads this per week, so a lost lower bound would also hide a person on dates before the week asked about. **Suggested fix:** Give Sofia one more standby date before the range, say 2026-10-05, which the range test must not return. The other tests ask for Ana and Luis, or for Sofia under another business, so they need no change. Test only, no code change. **Resolution:** Fixed 2026-10-01 in 5b.3's review-fix commit: Sofia also has 2026-10-05, before the range the test asks for. With the lower bound removed from findStandbyDates the range test now fails; with it back, all four pass. Closed 2026-10-01 by the feature 5b final independent review (53189a8..f5116a1): the fixture (find-standby-dates.test.ts:39) gives Sofia 2026-10-05, and the range test (line 56) asks for 2026-10-12 to 2026-10-19 and checks the exact result with `toEqual`, so a dropped `gte(standbyDate.date, fromDate)` (find-standby-dates.ts:29) would return 10-05 and fail it. The other three tests ask for Ana and Luis, an empty list, or another business, so the extra row changes none of them. The function is unchanged and correct; all 204 backend tests pass. Nothing new introduced.

Independent review

Status: passed **Target commit:** f5116a17fb811224978e2db76763b942e2e58d8f **Base commit:** 53189a8b50f0fda01c51b355876bc8b3cc472756 **Base ref:** main **Spec hash:** 09bfaa753af9061a5824c9ad57c5d560c5371a3004dd0413051a1e5e9cc88986 **Prepared by:** claude **Builder model:** claude-opus-5-5 **Requested reviewer:** claude **Requested model:** claude-opus-5-5 **Requested execution:** automatic **Requested at:** 2026-10-02T02:49:22Z **Workflow:** regular **Check required:** no **Reviewer adapter:** claude **Reviewer model:** claude-opus-5-5 **Reviewer context:** fresh subagent **Actual execution:** automatic **Reviewed at:** 2026-10-02T02:53:41Z **Scope:** current **Lenses:** quality, security, performance, tests **Verdict:** passed **Check result:** not-required

Commands

- `npm run build --workspace=backend` : pass
- `npm run build --workspace=frontend` : pass

- `npm run test --workspace=@scheduleads-app/shared` : pass (9 files, 66 tests)
- `npm run test --workspace=backend` : pass (19 files, 204 tests, local scheduleads_dev)
- `npm run format:check` : pass
- `npm run db:generate --workspace=@scheduleads-app/shared` : pass (no schema changes, no files written)
- `npm run db:seed --workspace=@scheduleads-app/shared` , run twice: pass (both runs report the two businesses already there and nothing made)

Evidence

- Freshness: HEAD, `git merge-base main HEAD` , the spec's SHA-256 and a clean tree apart from this file all matched the request before the review was written.
- Whole delta 53189a8..f5116a1 read (20 files): two tables, migrations 0010 and 0011 with journal and snapshots, the `booking_link` unique, `findServiceResources` , `findStandbyDates` , four test files, the seed, and two comment-only edits.
- Contracts match the spec: named primary keys, the service foreign key cascades, the person or place foreign key is `no action` , and 0010 creates the hand-moved unique before the foreign key that needs it.
- `findServiceResources` : `null` for a missing, inactive or other business's service; nobody ticked gives every active person; ticked but all inactive gives `[]` , never anyone; no place ticked gives `null` ; ordered by name then id; failures carry only `safeErrorReason` .
- `findStandbyDates` : an empty list answers `[]` without a query; the range includes both ends; scoped by business; ordered by date then resource; failures carry only a safe reason.
- Security: every read filters by `organizationId` , and the composite foreign keys refuse cross-business rows (asserted by constraint name in the rules tests). Performance: both reads use the primary keys `(bookingLinkId, resourceId)` and `(resourceId, date)` .
- Tests: every Done-when rule is asserted, no skipped or focused tests. F-57 re-examined and closed.

Findings

- None new. F-57 closed. F-52 (open, P3, for 5d), F-56 (unverified, P3), F-32 and F-47 (earlier features) unchanged.

Remaining risk

- Browser behaviour not checked: this feature has no visible screen, and Check was not required.
- F-56 stays unverified: a database on another machine that applied the first 0010 needs rebuilding before `db:migrate` .